4220 PROJECT · AUTHLAB

Beyond TOTP

同步通行金鑰與硬體安全金鑰的釣魚抵抗與帳號復原評估

專案導覽與實作說明

給協作者的理解指南

這個專案不是一個產品，而是一個「可重現的認證安全實驗室 + 研究報告」。
它用協定分析與受控實驗，比較密碼 + TOTP、synced passkey、硬體 FIDO2 key。

文件版本：2026-11-09

專案路徑：/Users/janezheng/Downloads/4220Project

技術棧：Python 3.11 · Flask · WebAuthn · Playwright · Chrome CDP

主要文件：docs/REPORT.md · docs/SUMMARY_zh.md · docs/slides.md

目錄

> **給 PARTNER 的閱讀建議**
>
> **只有 10 分鐘：**讀 §1–§3，先建立「這個專案在問什麼」的框架。
>
> **想看結論：**直接讀 §7 的結果表與 §8 的心智模型。
>
> **要跑程式：**看 §10，照著指令做即可；不想跑也可以直接讀已 checked-in 的結果。

01 這個專案在做什麼

02 核心研究問題

03 為什麼這三個方法值得比較

04 專案怎麼回答：三條主線

05 AuthLab 的架構

06 實驗設計：25 個情境

07 主要結果與怎麼讀

08 用什麼心智模型理解

09 程式碼導覽

10 怎麼運行這些程式

11 常見問題

12 最後：三個重點

本文件是專案的中文導覽，完整論證、參考文獻與實驗細節請看 `docs/REPORT.md`。 所有數字均來自 checked-in 的 `experiments/results/results.json`； 人類研究僅提供可執行協議，沒有人類受試者資料。

1. 這個專案在做什麼

一句話：**這是一個「可重現的認證安全實驗室 + 研究報告」，用來比較三種登入方式在釣魚、重放、裝置遺失與帳號復原下的實際表現。**

它不是產品，也不是要取代 Google/Microsoft 的登入系統；它的價值在於把每個安全決策拆開，讓你可以實際攻擊、觀察、量測，而不是只讀規格。

專案比較三種登入方式：

方法	憑證本質	主要風險
密碼 + TOTP	共享秘密 + 6 位數一次性碼	即時釣魚轉送、重放、共享秘密外洩
Synced passkey	WebAuthn 平台憑證，可透過同步機制到其他裝置	RP 驗證錯誤、同步帳號被盜、復原流程
硬體 FIDO2 key	不可匯出的 roaming 憑證	遺失即鎖死、備用金鑰管理、counter 限制

然後問五個問題：

1. **釣魚抵抗：**攻擊者能不能即時代理登入？
2. **重放保護：**攔截到的 TOTP 或 WebAuthn assertion 能不能再用一次？
3. **帳號復原：**手機丟了、金鑰丟了，使用者怎麼回來？
4. **同步與備份：**passkey 同步、備用金鑰、復原碼各自代表什麼交換？
5. **可用性：**這些安全機制對人類時間與認知負擔的影響有多大？

2. 核心研究問題

這個專案的核心問題不是「哪一種登入方式最快」，而是：

> 研究問題
>
> **當帳號面臨即時釣魚、重放攻擊、裝置遺失與復原流程濫用時，哪一種設計仍然站得住腳？**

速度當然重要，但它是第二位的。一個每天快 10 秒、卻在釣魚攻擊下完全失守的設計，不能稱為安全。因此，專案把「攻擊下的行為」放在「平均登入時間」之前，並用實驗分別量測兩者。

3. 為什麼這三個方法值得比較

3.1 TOTP 的問題不是「密碼學被破解」

TOTP 是共享秘密：伺服器和使用者都有同一把 seed。攻擊者只要架一個反向代理，把密碼和當前 TOTP 即時轉送給真正的伺服器，就能拿到 session。

伺服器端保存 `totp_last_step` 可以阻止**第二次使用**同一個碼，但阻止不了**第一次即時轉送**。這是整個專案最重要的切入點之一。

3.2 Passkey / 硬體金鑰的關鍵是 origin binding

WebAuthn 的簽章內容包含 `clientDataJSON`，裡面有瀏覽器實際所在的 origin。RP 必須自己檢查：

- `clientDataJSON.origin`
- `rpIdHash`
- challenge
- user verification
- counter（如果適用）

如果 RP 只驗簽章，卻信任 `clientDataJSON` 裡攻擊者送來的 origin，那麼同一把金鑰、同一個簽章，就會從「釣魚-resistant」變成「可被轉送」。

3.3 復原是最弱的一環

再強的登入方式，只要復原流程是「寄信到信箱」或「輸入一組可以被釣魚的復原碼」，攻擊者就會改攻復原流程。所以專案把 recovery 當成一等公民來測試。

4. 專案怎麼回答：三條主線

4.1 協定分析

`docs/protocol_analysis.md` 和 `docs/threat_model.md` 從訊息層級拆解：

- TOTP 的 seed、時間步、視窗、server-side replay counter
- WebAuthn 的 registration/authentication ceremony
- challenge、origin、RP ID、UV、counter 的角色
- recovery code、email token、session revocation 的順序與不變量

4.2 受控實驗

AuthLab 是一個 Flask 寫的 WebAuthn RP，搭配：

- SQLite
- scrypt 密碼與復原碼雜湊
- RFC 6238 TOTP
- WebAuthn（`py_webauthn`）
- session、節流、稽核紀錄
- 一次性復原碼
- Email 重設權杖
- 故意留給實驗用的 lab endpoints

實驗用 **Chrome + Playwright + CDP 虛擬認證器**，所以不需要實體金鑰。另有一個故意惡意的反向代理 `evil.app.lvh.me:5011`，用來做即時釣魚。

4.3 可用性

- **U1**：量測登入 ceremony 的實際延遲。
- **U2**：量測錯誤路徑（錯密碼、錯 TOTP、沒有憑證）的回饋與時間。
- **KLM**：用 Keystroke-Level Model 估算人類任務時間。
- **人類研究**：`survey/` 提供完整協議、SUS/NASA-TLX/信任問卷與分析管線，但**沒有捏造人類受試者資料**。

5. AuthLab 的架構

```
受害者瀏覽器
   │
   │  http://evil.app.lvh.me:5011
   ▼
釣魚代理（experiments/phishing_server.py）
   │  轉送密碼、TOTP、WebAuthn assertion
   ▼
真正的 RP（AuthLab，app.lvh.me:5010）
   │
   └─ SQLite
        users, credentials, sessions, flows,
        recovery_codes, recovery_tokens, throttle, audit_log
```

5.1 為什麼是 app.lvh.me 和 evil.app.lvh.me？

- `lvh.me` 是公開 DNS，會解析到 `127.0.0.1`。
- macOS 不解析 `*.localhost`，所以不用 `foo.localhost`。
- `app.lvh.me` 是 `evil.app.lvh.me` 的後綴，所以瀏覽器允許在 evil origin 使用 RP ID `app.lvh.me`，但簽章裡的 origin 仍然是 `http://evil.app.lvh.me:5011`。

這正好製造出「瀏覽器願意簽章，但 RP 必須拒絕」的情境。

5.2 兩個驗證器，差一個檢查就差很多

```
strict verifier：
  origin == http://app.lvh.me:5010 ?
    是 → 接受
    否 → 拒絕

permissive verifier（故意寫錯）：
  信任 clientDataJSON.origin
    → 攻擊者送什麼就信什麼 → 接受
```

這就是 P3/P4、P6/P7 的核心。

5.3 復原流程

- **復原碼**：10 組、每組 8 字元、約 40 bits、scrypt 雜湊、單次使用。
- **Email 重設**：32 bytes token、SHA-256 儲存、15 分鐘、單次使用、原子消耗。

- 重設成功後：撤銷所有 session、輪替 TOTP secret、刪除舊復原碼。
- 密碼變更後：撤銷所有舊 session，並發一個新的 recovery session。

6. 實驗設計：25 個情境

類別	數量	內容
P：Phishing	10	P1、P1b、P1c、P1d、P2–P7
R：Replay / counter	7	R1–R7
S：Sync / loss / recovery	8	S1–S8
U：Usability	2	U1 latency、U2 error paths
Protocol checks	1 組	cookie flags、challenge entropy、token hashing

每個情境 3–5 次 trial。控制方式：

1. 每個 trial 用全新的暫存 SQLite 資料庫。
2. 啟動全新的 Flask app。
3. 啟動真正的 RP 與釣魚代理。
4. 建立全新的瀏覽器 context 與虛擬認證器。
5. 註冊隨機使用者、固定測試密碼。
6. 註冊所需的憑證。
7. 執行攻擊或防禦流程。
8. 記錄成功/失敗、時間、錯誤訊息。

統計上用 **Wilson 95% 信賴區間**。因為 n 只有 3–5，區間很寬，所以結論是「協定層級的定性結果」，不是母體比例估計。

6.1 實驗環境與 provenance

> CHECKED-IN RUN
>
> macOS 15.7.7（arm64）、Python 3.11.1、Google Chrome 156.0.8078.80、Playwright 1.63.0、py_webauthn 3.0.1、Flask 3.x、SQLite（WAL）。 25 個情境，執行時間 **345.85 秒**。
>
> `experiments/results/results.json` 的 SHA-256：
> `0553d4be5ea16dd04520567b700fe4863b3812300a7747eac15a708b779c3fe4`

6.2 這個專案不是什麼（限制）

- 使用 Chrome CDP 虛擬認證器，**不是實體金鑰**；不模擬廠商雲端同步的內部實作。
- 所有伺服器在 loopback，**沒有 WAN 延遲與真實郵件遞送**。
- 每個情境 3–5 次，Wilson 信賴區間很寬；結果是協定層級的定性結論，不是母體比例估計。

- 僅使用 Chrome；未測試 Firefox/Safari 的差異。
- **沒有人類受試者資料**；可用性部分是 KLM 分析模型與質性比較。
- 未進行形式化驗證（例如 ProVerif/Tamarin）。

> **安全提醒**
>
> `experiments/phishing_server.py` 是**故意惡意的代理**。它只綁定 loopback、只用丟棄式測試帳號，**絕對不要**把它暴露到網路，也不要用它指向真實服務。詳見 `docs/ethics.md`。

7. 主要結果與怎麼讀

7.1 先講一個很容易誤讀的地方

`experiments/results/summary.md` 的 `Observed` 欄位是「安全相關結果」：

- 對**攻擊情境**：數字是攻擊成功率 → 越低越好。
- 對**防禦情境**：數字是防禦成功率（trials − attack successes）→ 越高越好。

所以：

情境	數字	意思
P1 密碼 + TOTP 即時轉送	5/5	攻擊成功 5/5 → **壞**
P1b 重放 TOTP	5/5	防禦成功 5/5 → **好**
P2 passkey lookalike	0/3	攻擊成功 0/3 → **好**
P4 passkey permissive relay	5/5	攻擊成功 5/5 → **壞**
S2 平台憑證遺失	3/3	鎖死 3/3 → **預期中的損失結果**

表格裡有 `Direction` 欄位，raw JSON 也有 `success_meaning`，讀的時候先看那個。

7.2 釣魚

情境	結果	解讀
P1 密碼 + TOTP 即時轉送	5/5 成功	TOTP 無法抵抗即時 relay
P1b 事後重放 TOTP	5/5 被阻擋	server-side counter 有效，但救不了第一次
P1c 過期 TOTP（±2 步）	3/3 被拒	視窗有邊界
P1d 無狀態 TOTP 驗證器	5/5 可用兩次	沒有 `totp_last_step` 就沒有重放保護
P2 / P5 naive lookalike	0/3	攻擊者用自己的 RP ID，瀏覽器找不到憑證
P3 / P6 嚴格 origin relay	0/5	RP 正確檢查 origin → 拒絕
P4 / P7 寬鬆 origin relay	5/5	RP 信任 `clientDataJSON.origin` → 被攻破

> 核心結論
>
> **WebAuthn 的釣魚抵抗不是憑證自動帶來的，而是「正確的 RP 驗證」帶來的。**

7.3 重放與 counter

- TOTP 視窗：−1/0/+1 接受，±2 拒絕。
- 捕獲的 WebAuthn assertion：同 flow 重放、換新 flow 重放都 3/3 被拒（flow 單次使用 + challenge 新鮮度）。
- 硬體金鑰「落後」的克隆：3/3 被拒（counter regression）。
- 硬體金鑰「超前」的偽造 counter：3/3 被接受（只能偵測落後，不能偵測超前）。
- Synced passkey 的非單調/零 counter：3/3 被接受（跨裝置同步的相容性代價）。

7.4 裝置遺失與復原

情境	結果
S1 遺失裝置 A，從同步還原到 B	5/5 成功
S2 遺失 device-bound 平台憑證	3/3 鎖死
S3 遺失唯一硬體金鑰	3/3 鎖死
S4 使用預先註冊的備用硬體金鑰	3/3 成功
S5 硬體金鑰遺失後使用復原碼	3/3 成功
S6 重複使用復原碼	3/3 被拒
S7 Email 重設	3/3 全部不變量成立
S8 過期重設權杖	3/3 被拒

7.5 可用性

實際量測（不含人類思考與輸入）：

任務	中位數
密碼 + TOTP 登入	158 ms
Passkey 登入	103 ms
硬體金鑰登入	100 ms
復原碼登入	94 ms
Email 重設送出	42 ms

KLM 人類任務時間估算：

任務	估算
密碼 + TOTP	13.5 s
Synced passkey	3.5 s
硬體金鑰	4.0 s
復原碼	9.2 s
Email 重設	18.2 s（不含郵件遞送）

注意：KLM 是分析模型，不是人類實測。真正的問卷與實驗協議在 `survey/`，但沒有執行、也沒有捏造資料。

8. 用什麼心智模型理解這個專案

建議用三層來看：

第一層：憑證層

攻擊者能拿到什麼？

- **TOTP**：共享秘密，可以被即時轉送。
- **Passkey**：私鑰在平台/sync fabric 裡，正常情況下不可直接匯出；但同步機制讓它可以在多台裝置使用。
- **硬體金鑰**：私鑰不可匯出，這是最強的「防複製」屬性。

第二層：驗證層

伺服器到底檢查了什麼？

- **TOTP**：`totp_last_step`、時間視窗、節流。
- **WebAuthn**：origin、RP ID、challenge、UV、counter、flow 單次使用。
- 這一層才是 P3/P4、P6/P7 的病根。

第三層：復原層

失去裝置後，系統怎麼讓使用者回來？

- **Synced passkey**：靠同步 → 可用性高，但同步帳號成為新攻擊面。
- **硬體金鑰**：靠備用金鑰或復原碼 → 沒有備份就鎖死。
- **Email reset**：靠信箱 → 信箱被盜就等於帳號被盜。
- **復原碼**：一次性、短效、原子消耗、撤銷、輪替、節流，缺一不可。

> **一句話總結**
>
> **帳號安全不是「用了哪一種憑證」而已，而是「憑證 + 驗證邏輯 + 復原流程 + 營運政策」一起決定的系統性質。**

9. 程式碼導覽

路徑	內容	建議
`authlab/security.py`	scrypt、TOTP、session、flow、復原碼	先看這個，理解核心 primitive
`authlab/webauthn_service.py`	`py_webauthn` 包裝、counter policy	看 WebAuthn 參數與驗證
`authlab/routes/auth.py`	註冊、密碼 + TOTP 登入、節流	看兩步驟登入
`authlab/routes/passkeys.py`	WebAuthn 註冊/登入、strict/permissive 驗證	看 P3/P4 的實作
`authlab/routes/recovery.py`	復原碼、Email reset、撤銷	看 recovery 不變量
`authlab/routes/lab.py`	只有 lab mode 開啟的測試端點	看實驗如何取得 ground truth
`authlab/static/app.js`	瀏覽器端 WebAuthn 序列化	看 assertion 怎麼送出
`experiments/harness.py`	Playwright/CDP、Lab lifecycle、helpers	看實驗怎麼控制瀏覽器
`experiments/phishing_server.py`	惡意反向代理	看即時 relay 怎麼做
`experiments/run_experiments.py`	25 個情境	看每個 scenario 的攻擊步驟
`experiments/analyze.py`	Wilson CI、latency、KLM、圖表	看結果怎麼變成表格
`docs/REPORT.md`	完整英文報告	主要閱讀入口
`docs/SUMMARY_zh.md`	繁體中文摘要	10 分鐘快速理解
`docs/protocol_analysis.md`	訊息層級分析	想深入協定時看
`docs/threat_model.md`	資產、對手、信任邊界	想理解威脅模型時看
`docs/experiment_design.md`	變數、控制、效度	想 critique 實驗時看
`survey/`	人類研究協議與分析	問卷與 SUS/TLX 分析

建議閱讀順序

1. `README.md`：先知道怎麼跑。
2. `docs/SUMMARY_zh.md`：用中文快速掌握結論。
3. `docs/REPORT.md`：完整論證與結果。
4. `docs/protocol_analysis.md`：訊息層級細節。
5. `docs/threat_model.md`：對手模型與信任邊界。
6. `docs/experiment_design.md`：實驗設計與效度威脅。
7. 程式碼：`security.py` → `webauthn_service.py` → routes → harness。
8. `docs/slides.md`：如果要報告，這份是 12–15 分鐘的 deck。

10. 怎麼運行這些程式

10.1 前置需求

- Python 3.11+
- Google Chrome（實驗用系統 Chrome，不是內建 Chromium）
- macOS 或 Linux
- 不需要 Node/npm

先確認：

```
cd /Users/janezheng/Downloads/4220Project
python3 --version
ls "/Applications/Google Chrome.app"
```

安裝套件：

```
python3 -m pip install -r requirements.txt
```

建議用 virtualenv，但這個專案沒有強制。

10.2 正常模式：自己手動玩 AuthLab

```
cd /Users/janezheng/Downloads/4220Project
python3 run.py
```

然後開：

```
http://localhost:5010
```

> **重要**
> 不要用 `http://127.0.0.1:5010`，因為預設 `RP_ID=localhost`，WebAuthn RP ID 必須是 origin host 的後綴。

正常模式下：

- 註冊帳號
- 掃 TOTP QR code（Google Authenticator、1Password 等）
- 註冊 passkey（macOS 上可用 Touch ID）
- 或註冊硬體 FIDO2 key
- 測試登入、復原碼、Email reset

正常模式 `AUTHLAB_LAB=0`，所以 `/lab/*` 端點會回 404。這是故意的。

停止：`Ctrl+C`。

10.3 實驗模式：一鍵跑完整套件

實驗會自己啟動伺服器，不需要先開 `run.py`。先清掉佔用埠的程式：

```
cd /Users/janezheng/Downloads/4220Project
pkill -f "run.py" || true
```

跑實驗：

```
python3 experiments/run_experiments.py
```

跑分析：

```
python3 experiments/analyze.py
```

實驗模式會：

- 在 `app.lvh.me:5010` 啟動真正的 RP
- 在 `evil.app.lvh.me:5011` 啟動釣魚代理
- 使用暫存 SQLite
- 啟動 headless Chrome + CDP 虛擬認證器
- 執行 25 個情境
- 把結果寫到 `experiments/results/results.json`

大約 **6–10 分鐘**（這台機器實測 345.85 秒）。

10.4 如果 app.lvh.me 無法解析

正常情況下 `lvh.me` 會透過公開 DNS 解析到 `127.0.0.1`。如果沒有：

```
sudo sh -c 'echo "127.0.0.1 app.lvh.me evil.app.lvh.me" >> /etc/hosts'
```

確認：

```
ping -c 1 app.lvh.me
ping -c 1 evil.app.lvh.me
```

10.5 如果 Chrome 找不到

Harness 使用 `channel="chrome"`，所以必須安裝 Google Chrome。路徑通常是：

```
/Applications/Google Chrome.app
```

10.6 看結果

分析完成後會產生：

```
experiments/results/results.json    原始機器可讀資料
experiments/results/summary.md      25 個情境 + Wilson CI
experiments/results/latency.csv     每個任務的延遲
experiments/results/latency.md      延遲表格
experiments/results/klm.md          KLM 分析
experiments/results/analysis.json   分析後的結構化資料
figures/phishing.png                釣魚結果
figures/replay_recovery.png         重放與復原
figures/latency.png                 延遲
figures/klm.png                     KLM
figures/totp_window.png             TOTP 視窗
```

最快的方式：

```
cat experiments/results/summary.md
cat experiments/results/latency.md
cat experiments/results/klm.md
open figures/phishing.png
```

原始資料的 SHA-256：

```
shasum -a 256 experiments/results/results.json
```

目前 checked-in 版本是：

```
0553d4be5ea16dd04520567b700fe4863b3812300a7747eac15a708b779c3fe4
```

10.7 只跑單一情境（進階除錯）

如果你想只跑 P4（passkey permissive relay），可以這樣：

```
cd /Users/janezheng/Downloads/4220Project
python3 - <<'PY'
from experiments.harness import Lab
from experiments.run_experiments import scenario_p4_passkey_permissive

with Lab() as lab:
    result = scenario_p4_passkey_permissive(lab)
    print(result["id"], result["successes"], "/", result["trials"])
PY
```

`Lab()` 會自己啟動兩個 server、Chrome、暫存 DB，結束後清理。

注意：目前 harness 固定 `headless=True`，所以你不會看到瀏覽器視窗；這是正常的。

10.8 問卷分析

先跑內建的 synthetic self-test：

```
python3 survey/analyze_survey.py --selftest
```

這會用**明確標示的合成資料**跑完整分析流程，不會讀寫真實研究資料。

如果你真的有受試者資料，依照 `survey/responses_template.csv` 準備 CSV，然後：

```
python3 survey/analyze_survey.py path/to/responses.csv --out survey/results
```

輸出的統計包含：

- SUS
- NASA-TLX
- trust / perceived security
- Friedman test
- pairwise Wilcoxon + Holm correction
- Cochran's Q / McNemar
- effect sizes

再次強調：這個 repo **沒有**捏造的人類資料，`--selftest` 的資料是合成且標示清楚的。

11. 常見問題

問題	原因 / 解法
`Address already in use`	先 `pkill -f "run.py"`，或檢查 `lsof -nP -iTCP:5010 -sTCP:LISTEN`
WebAuthn 在 `127.0.0.1` 失敗	用 `http://localhost:5010`，因為 `RP_ID=localhost`
`/lab/health` 回 404	正常模式關閉 lab endpoints，這是預期行為
`app.lvh.me` 無法解析	加 `/etc/hosts`，或確認 `lvh.me` DNS
實驗跑很久	第一次跑、Chrome 啟動、25 個情境；約 6–10 分鐘
實驗結果的 latency 每次不同	正常，latency 取決於機器；協定結果應該是穩定的
想看瀏覽器畫面	目前 harness 固定 headless，需自行改 `Lab.__enter__` 的 `headless=True`
沒有實體金鑰	實驗不需要；正常模式才需要真的 passkey/硬體金鑰
`.env.example` 會自動載入嗎？	不會。`python3 run.py` 只讀環境變數；`.env.example` 只是範例，請自行 `export` 或使用 shell 設定

12. 最後：三個重點

如果你只記三件事：

1. **TOTP 的重放保護只防第二次，不防第一次即時轉送。**
2. **Passkey 與硬體金鑰的釣魚抵抗，取決於 RP 是否正確檢查 origin；不是憑證本身自動保證。**
3. **復原流程是最弱的一環；sync、備用金鑰、復原碼、Email reset 都會重新打開攻擊面。**

而這個專案用可重現的程式碼與實驗，把這三件事變成可以自己跑、自己驗證的結果，而不是只有文獻上的主張。

> 給 PARTNER 的一句話
>
> 這個專案的核心不是「passkey 比 TOTP 好」，而是：**同一種憑證，遇到不同的驗證邏輯與復原設計，會得到完全不同的安全結果。** 報告裡的每一個數字，都是為了證明這件事。

附錄 A：快速指令卡

把以下指令當作 cheat sheet；完整說明在 §10。

```
# 安裝
python3 -m pip install -r requirements.txt
# 正常模式 → http://localhost:5010
python3 run.py
# 完整實驗（約 6–10 分鐘）
pkill -f "run.py" || true
python3 experiments/run_experiments.py
python3 experiments/analyze.py
# 問卷分析（合成資料自我測試）
python3 survey/analyze_survey.py --selftest
```

附錄 B：名詞速查

名詞	意思
RP / RP ID	Relying Party 是依賴方（AuthLab）；RP ID 是 WebAuthn 允許的網域後綴（`app.lvh.me`）。
Origin	瀏覽器實際所在的 scheme + host + port；strict 驗證的重點。
Challenge	每次 ceremony 的隨機挑戰，用來防重放。
UV	User Verification，PIN 或生物辨識。
Counter	Authenticator 的簽章計數器；硬體金鑰嚴格單調，synced passkey 常放寬。
Synced passkey	可透過平台同步的 passkey；以 backup eligible/state 表示。
TOTP / Replay	時間為基礎的一次性密碼；重放是重複使用已捕獲的 TOTP 或 assertion。
KLM / Wilson CI	KLM 是分析式可用性估算；Wilson CI 是小樣本比例的信賴區間（95%）。